

Guia de Integração e Publicação

Painel de Acompanhamento — Revisões e Tacógrafos

Este guia detalha o passo a passo para:

- Conectar suas planilhas Excel Online (SharePoint/OneDrive) ao painel via Microsoft Graph API
- Publicar o painel como site acessível no GitHub Pages

Pré-requisitos: conta Microsoft 365, acesso ao Azure Portal, conta GitHub.

PARTE 1 — Conexão com as Bases Excel via Microsoft Graph API

A Microsoft Graph API permite ler planilhas do SharePoint/OneDrive diretamente via HTTP. O painel fará requisições para buscar os dados atualizados toda vez que for aberto.

Passo 1 — Registrar um aplicativo no Azure AD

1.1 — Acessar o portal

Acesse <https://portal.azure.com> com sua conta corporativa Microsoft 365.

1.2 — Criar o App Registration

Navegue até **Azure Active Directory** (ou Entra ID) → **App registrations** → **New registration**.

Preencha:

Campo	Valor
Name	Painel Manutenção Maroni
Supported account types	Accounts in this organizational directory only
Redirect URI (Web)	https://seuusuario.github.io/painel-manutencao/

1.3 — Anotar os IDs

Após criar, anote os dois valores que aparecem na página do app:

- **Application (client) ID** — ex: a1b2c3d4-e5f6-7890-abcd-1234567890ab
- **Directory (tenant) ID** — ex: f1e2d3c4-b5a6-7890-fedc-0987654321ba

Passo 2 — Configurar permissões da API

2.1 — Adicionar permissões

No menu lateral do app, vá em **API permissions** → **Add a permission** → **Microsoft Graph** → **Delegated permissions**.

Adicione estas permissões:

Permissão	Motivo
Files.Read.All	Ler arquivos do SharePoint/OneDrive
Sites.Read.All	Acessar sites do SharePoint
User.Read	Login do usuário (obrigatório)

2.2 — Conceder consentimento

Clique em **Grant admin consent for [sua org]**. Todos os checks devem ficar verdes.

Passo 3 — Habilitar fluxo implícito (SPA)

Como o painel é 100% frontend (sem backend), usaremos autenticação via SPA:

3.1 — Vá em **Authentication** → **Platform configurations** → **Add a platform** → **Single-page application**.

3.2 — Em Redirect URIs, adicione:

<https://seuusuario.github.io/painel-manutencao/>
<http://localhost:5500/> (para testes locais)

3.3 — Marque **Access tokens** e **ID tokens** em "Implicit grant and hybrid flows".

Passo 4 — Obter os IDs das planilhas

Você precisa dos IDs das duas planilhas no SharePoint/OneDrive. A forma mais fácil:

4.1 — Abra cada planilha no Excel Online (navegador).

4.2 — Na URL, identifique os parâmetros. Exemplo:

```
https://grupomaroni.sharepoint.com/:x:/s/PCM/EaBcDeFgHiJkLmNoPq?e=AbCdEf
```

4.3 — Alternativamente, use o Graph Explorer para descobrir:

Acesse <https://developer.microsoft.com/graph/graph-explorer>, faça login, e execute:

```
GET https://graph.microsoft.com/v1.0/me/drive/root/search(q='BASE DE REVISÕES')?select=id,name,webUrl
```

Anote o **id** de cada arquivo retornado. Você precisará de:

- **driveId** e **itemId** da planilha "BASE DE REVISÕES 3.0.1"
- **driveId** e **itemId** da planilha "GESTÃO DE CRONOTACÓGRAFO"

Passo 5 — Adicionar o código de integração ao painel

Adicione a biblioteca MSAL.js (autenticação Microsoft) e o código de leitura. Insira no **<head>** do HTML:

```
<script src="https://alcdn.msauth.net/browser/2.38.0/js/msal-browser.min.js"></script>
```

E adicione este bloco JavaScript antes do fechamento **</body>**:

```
const msalConfig = {
  auth: {
    clientId: "SEU_CLIENT_ID_AQUI",
    authority: "https://login.microsoftonline.com/SEU_TENANT_ID",
    redirectUri: window.location.origin + window.location.pathname
  }
};

const msalApp = new msal.PublicClientApplication(msalConfig);

async function getToken() {
  const accounts = msalApp.getAllAccounts();
  const request = { scopes: ["Files.Read.All", "Sites.Read.All"] };
  if (accounts.length > 0) {
    request.account = accounts[0];
    try { return (await msalApp.acquireTokenSilent(request)).accessToken; }
    catch { return (await msalApp.acquireTokenPopup(request)).accessToken; }
  }
  return (await msalApp.acquireTokenPopup(request)).accessToken;
}

async function fetchExcel(driveId, itemId, sheet, range) {
  const token = await getToken();
  const url = `https://graph.microsoft.com/v1.0/drives/${driveId}`
    + `/items/${itemId}/workbook/worksheets('${sheet}')`
    + `/range(address='${range}')?$select=values`;
  const res = await fetch(url, {
    headers: { Authorization: `Bearer ${token}` }
  });
  const data = await res.json();
  return data.values; // Array de arrays com os dados
}
```

Substitua **SEU_CLIENT_ID_AQUI** e **SEU_TENANT_ID** pelos valores do Passo 1.3.

Atenção: nunca exponha client secrets no frontend. O fluxo SPA usa apenas o client ID, que é seguro para código público.

Passo 6 — Chamar a API ao carregar o painel

Substitua os dados estáticos (const R = [...]) por chamadas dinâmicas:

```
async function carregarDados() {  
  // IDs das suas planilhas (obtidos no Passo 4)  
  const DRIVE_REV = "seu_drive_id_revisoes";  
  const ITEM_REV = "seu_item_id_revisoes";  
  const DRIVE_TAC = "seu_drive_id_tacografos";  
  const ITEM_TAC = "seu_item_id_tacografos";  
  
  // Buscar dados das revisões  
  const revRaw = await fetchExcel(  
    DRIVE_REV, ITEM_REV,  
    "Base de Revisões 3.0.1", "A1:Y7260"  
  );  
  
  // Buscar dados dos tacógrafos  
  const tacRaw = await fetchExcel(  
    DRIVE_TAC, ITEM_TAC,  
    "GESTÃO AFERIÇÃO", "A1:N7346"  
  );  
  
  // Aplicar filtros e popular as tabelas  
  R.length = 0;  
  R.push(...processarRevisoes(revRaw));  
  T.length = 0;  
  T.push(...processarTacografos(tacRaw));  
  filRev(); filTac(); dRev(); dTac();  
}  
  
carregarDados();
```

PARTE 2 — Publicar no GitHub Pages

Passo 1 — Criar o repositório

1.1 — Acesse <https://github.com/new> e crie um repositório:

Campo	Valor
Repository name	painel-manutencao
Visibility	Private (recomendado) ou Public
Initialize	Marcar "Add a README file"

Nota: repositórios privados com GitHub Pages requerem GitHub Pro, Team ou Enterprise. Caso contrário, use público.

Passo 2 — Fazer upload do painel

2.1 — No repositório criado, clique em **Add file** → **Upload files**.

2.2 — Arraste o arquivo **painel_v4.html** e renomeie para **index.html**.

2.3 — Clique em **Commit changes**.

Ou via terminal (se tiver Git instalado):

```
git clone https://github.com/seuusuario/painel-manutencao.git
cd painel-manutencao
cp ~/Downloads/painel_v4.html index.html
git add index.html
git commit -m "Adicionar painel de manutenção"
git push origin main
```

Passo 3 — Ativar o GitHub Pages

3.1 — No repositório, vá em **Settings** → **Pages** (menu lateral).

3.2 — Em "Source", selecione **Deploy from a branch**.

3.3 — Em "Branch", selecione **main** e pasta **/ (root)**.

3.4 — Clique em **Save**.

Após 1-2 minutos, seu painel estará acessível em:

```
https://seuusuario.github.io/painel-manutencao/
```

Passo 4 — Atualizar a Redirect URI no Azure

Volte ao Azure Portal → seu App Registration → **Authentication** e confirme que a URL do GitHub Pages está na lista de Redirect URIs. Sem isso, o login Microsoft não funciona.

PARTE 3 — Checklist Final

#	Etapa	Status
1	Registrar App no Azure AD (client ID + tenant ID)	☐
2	Configurar permissões Files.Read.All + Sites.Read.All	☐
3	Habilitar SPA + Redirect URIs	☐
4	Obter driveId + itemId de cada planilha via Graph Explorer	☐
5	Adicionar MSAL.js e código de integração ao HTML	☐
6	Testar localmente (Live Server no VS Code)	☐
7	Criar repositório no GitHub	☐
8	Upload do index.html + ativar GitHub Pages	☐
9	Atualizar Redirect URI no Azure com URL do GitHub Pages	☐
10	Teste final no navegador com login Microsoft	☐

Arquitetura Final

Componente	Tecnologia	Onde roda
Frontend (painel)	HTML + JS puro	GitHub Pages
Autenticação	MSAL.js (Microsoft)	Navegador do usuário
API de dados	Microsoft Graph API	Cloud Microsoft
Dados	Excel Online (SharePoint)	SharePoint/OneDrive

Vantagens desta abordagem:

- **Sem servidor** — tudo roda no navegador, sem custo de hospedagem
- **Dados sempre atualizados** — lê direto das planilhas a cada acesso
- **Seguro** — autenticação Microsoft, sem expor credenciais
- **Gratuito** — GitHub Pages + Graph API incluída no M365